



Sistemi Distribuiti

Laurea in Informatica

Michele Ciavotta

michele.ciavotta@unimib.it

Indice

Recap: Applicazioni multi-tier

- Modelli multi-tier vs. multi-layer
- Distribuzione dei compiti tra i layer

Applicazioni RIA (Rich Internet Application)

- Client programmabili con JavaScript (DHTML)
- Ajax (Asynchronous JavaScript and XML) - Interazioni asincrone con il server

Applicazioni di base di esempio

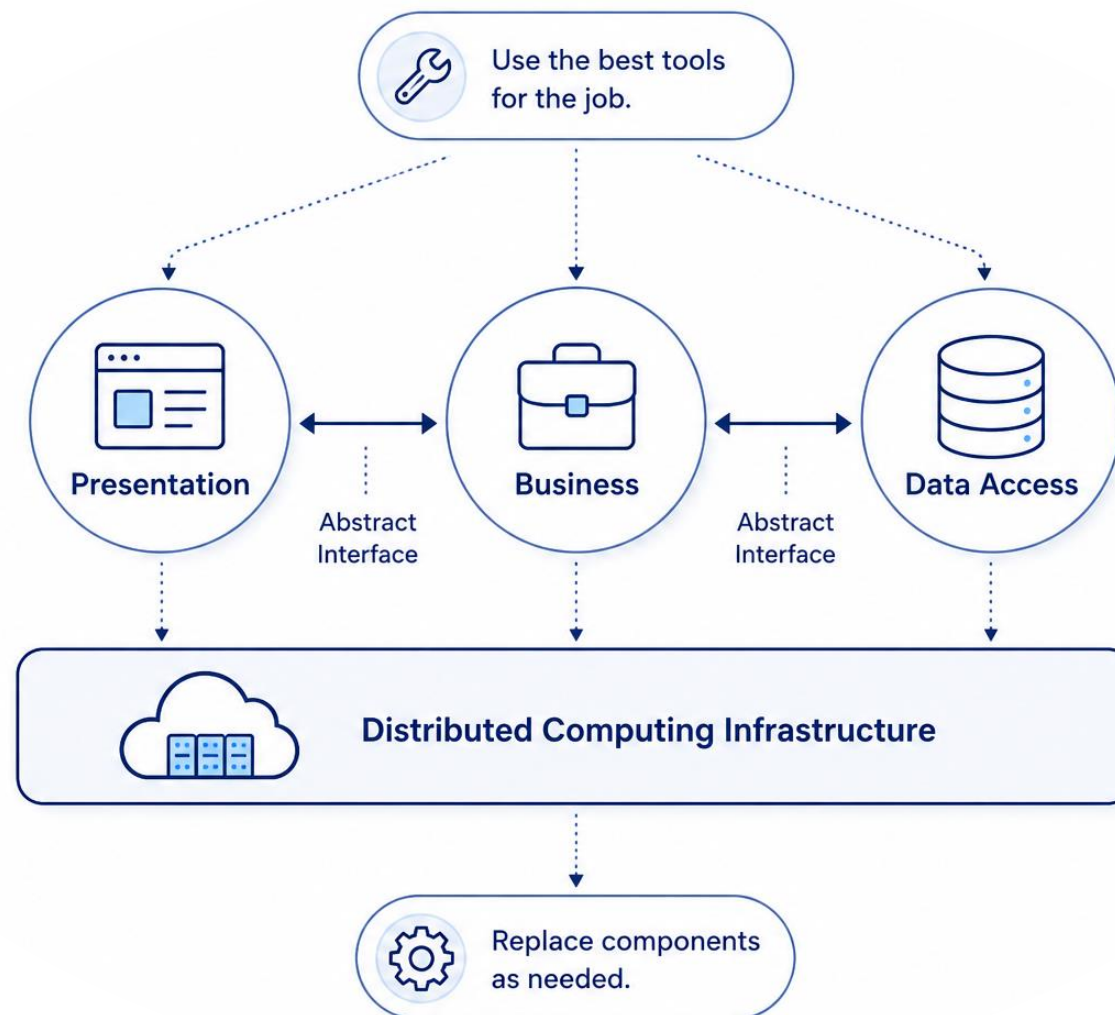
- Applicazioni solo lato client (browser)
- Applicazioni client-server (browser – Web server)

Architettura multi-tier

Un'applicazione è composta da componenti che collaborano per eseguire un task

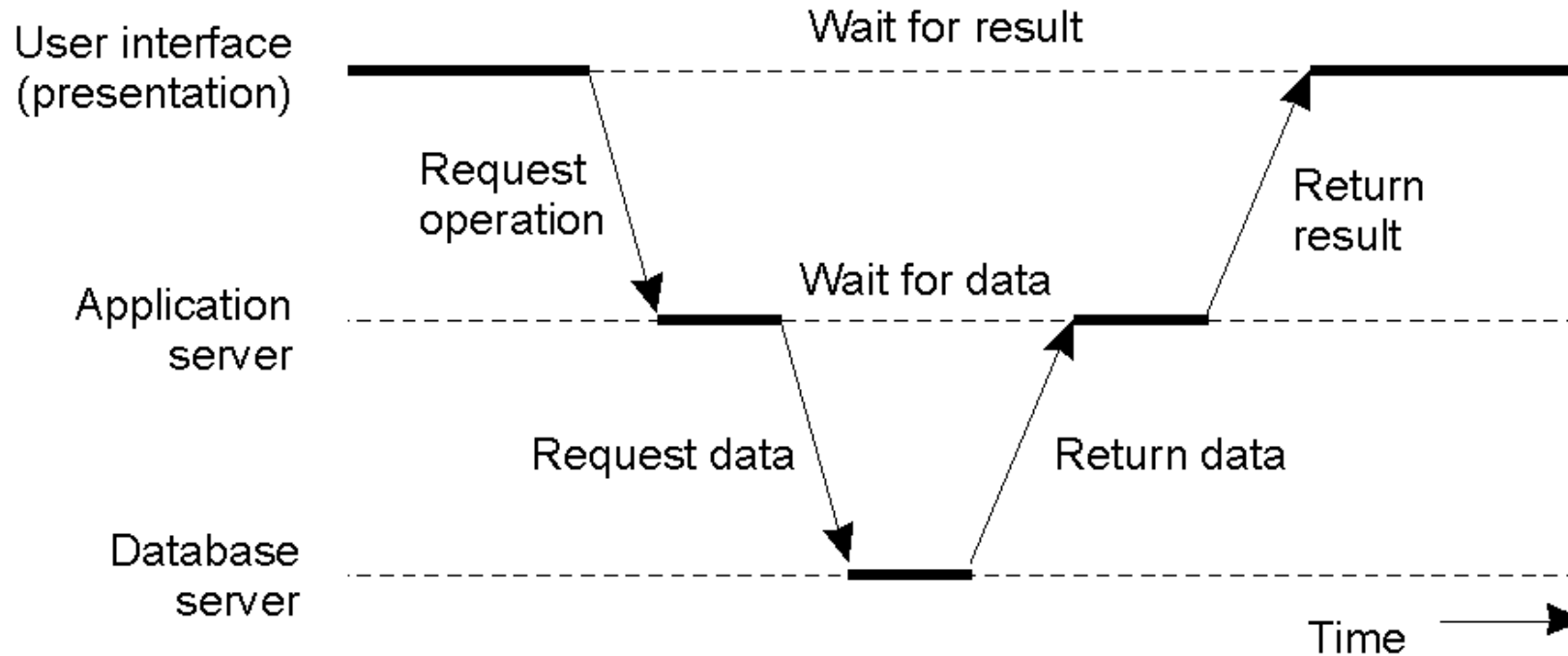
La decomposizione tipica è in tre livelli (MVC):

- Presentazione
- Logica
- Dati



Architettura multi-tier

In un'applicazione a tre tier, il tier di business funge da server per il tier di presentazione e da client per il tier dei dati



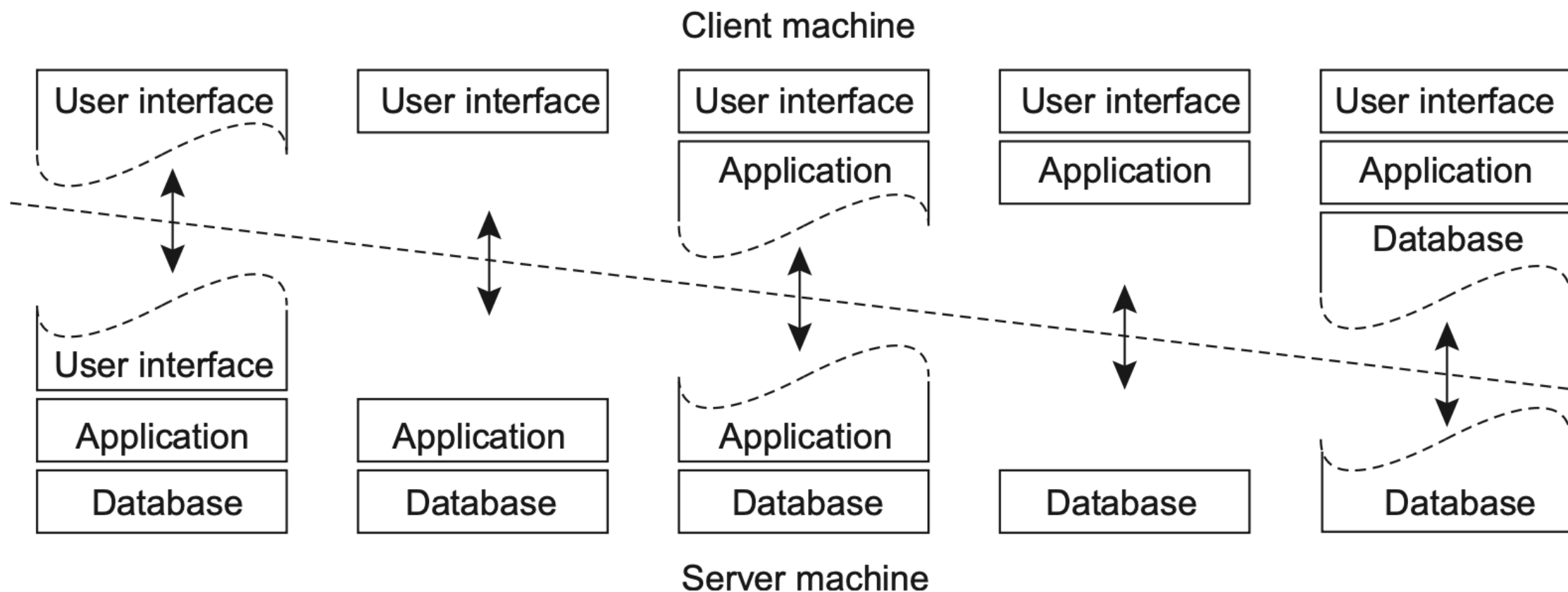
Il problema è come organizzare il sistema

- Ovvero, come distribuire i componenti tra macchine client/server, o cosa può essere implementato lato client e lato server

Da thin a fat client

L'applicazione distribuita si suddivide in tre livelli logici:

1. **interfaccia utente** (presentazione),
2. **elaborazione** (logica di business), e
3. **gestione dei dati** (database o file system).



DHTML - AJAX

RIA (RICH INTERNET APPLICATIONS)

Limiti del Web tradizionale

Il ciclo richiesta/risposta

- Ogni interazione dell'utente innesca una richiesta HTTP al server
- Il server risponde restituendo l'intera pagina HTML, non solo il dato richiesto
- Il browser ricarica completamente la pagina ad ogni risposta

Conseguenze per l'utente

- Attesa del caricamento completo anche per aggiornare un singolo dato
- Esperienza discontinua: la pagina "lampeggia" ad ogni interazione
- Traffico di rete sproporzionato rispetto all'informazione effettivamente nuova
- Interfacce poco reattive, distanti dall'esperienza delle applicazioni desktop

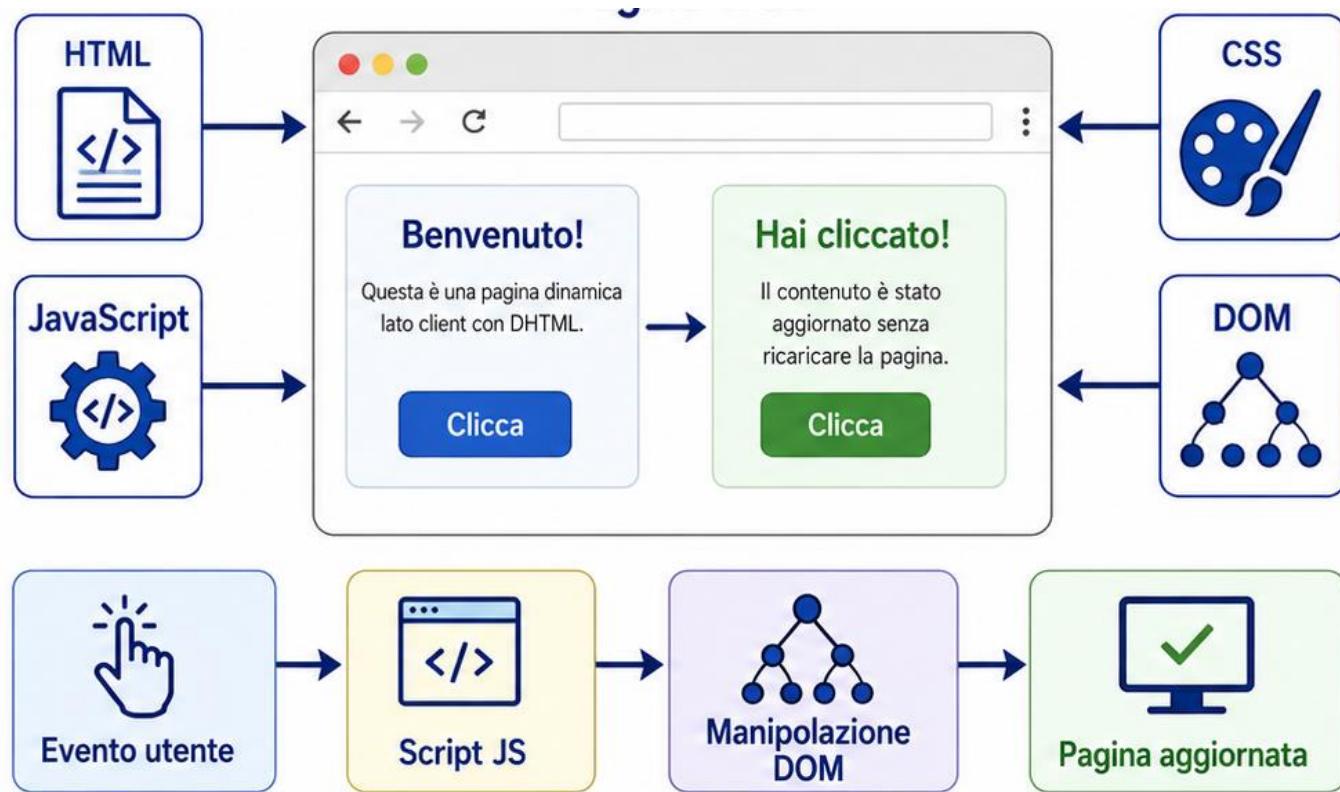
DHTML: Pagine web dinamiche lato client

Definizione — DHTML non è un linguaggio specifico, ma l'integrazione di HTML, CSS, JavaScript e DOM per rendere una pagina interattiva e dinamica lato client.

Componenti principali

- **HTML:** struttura dei contenuti
- **CSS:** presentazione e stile
- **JavaScript:** comportamento e logica
- **DOM:** accesso e manipolazione degli elementi della pagina

Permette di aggiornare il contenuto senza ricaricare la pagina, reagire agli eventi utente, modificare stile e struttura del documento, costruire interfacce più ricche e reattive.



Asynchronous JavaScript and XML (AJAX) : cos'è?

Limite di DHTML — può modificare contenuti e stile nel browser, ma da solo non introduce un meccanismo strutturato di comunicazione asincrona con il server.

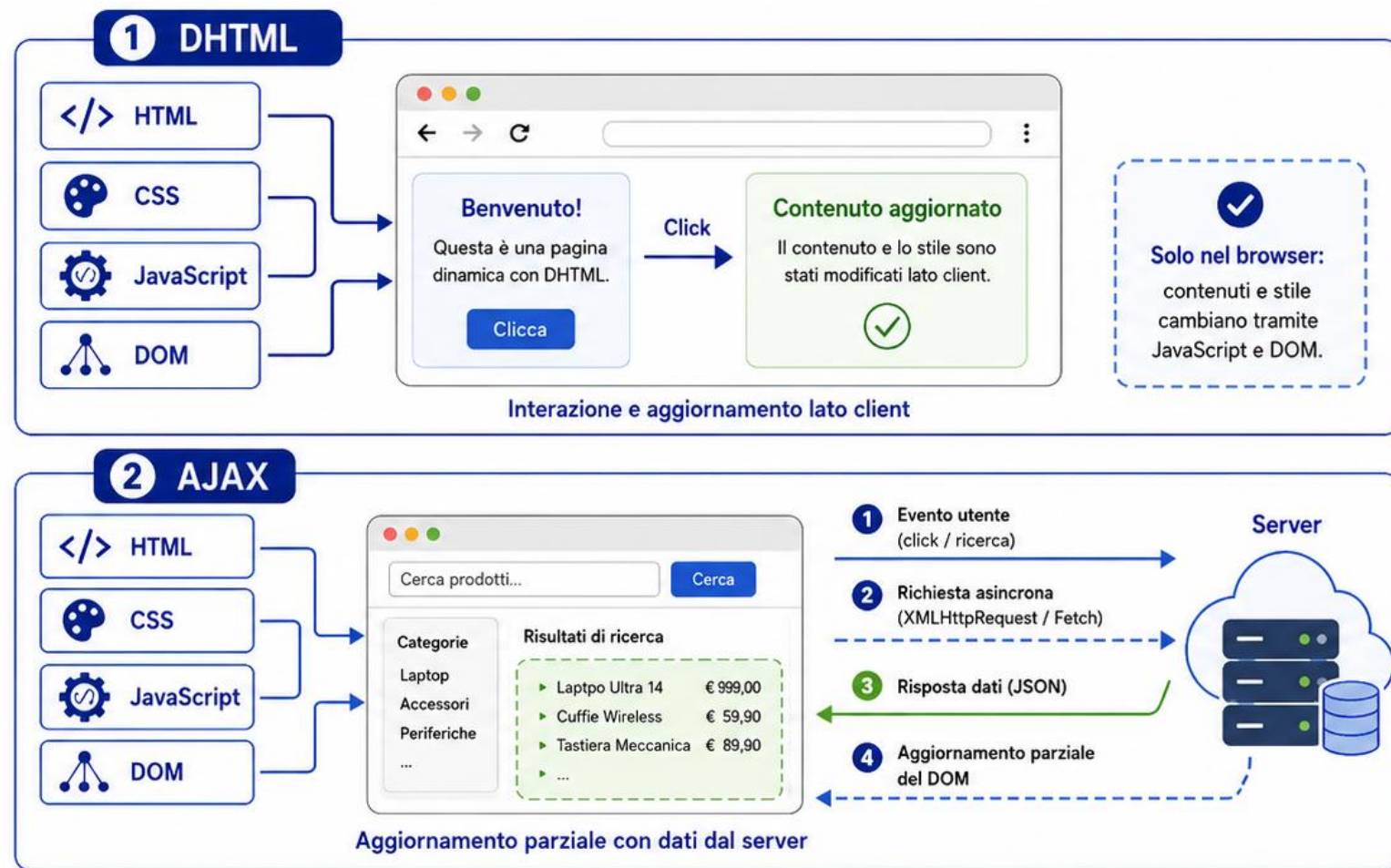
Asynchronous JavaScript and XML (AJAX)

- Modello per applicazioni web interattive (Jesse James Garrett, 2005)
- Non una nuova tecnologia, ma un **pattern** che combina componenti già esistenti

AJAX estende DHTML — aggiunge richieste asincrone (XMLHttpRequest o Fetch) verso il server, riceve dati (oggi tipicamente JSON) e aggiorna solo una parte della pagina senza ricaricarla.

Cosa abilita

- Aggiornamento parziale della pagina (solo i dati che cambiano)
- Interfacce *reattive*, vicine all'esperienza desktop (rich UI)
- Riduzione del traffico di rete e dei tempi di attesa



Web 2.0 e AJAX

AJAX è la tecnologia abilitante del **Web 2.0**, il paradigma definito da Tim O'Reilly nel 2005.

I sei principi del Web 2.0

- **Il Web come piattaforma** — le applicazioni vivono nel browser, non sul desktop
- **Intelligenza collettiva** — il valore cresce con il contributo degli utenti
- **Al di sopra del singolo dispositivo** — servizi accessibili da qualsiasi client
- **Servizi, non software in pacchetto** — rilascio continuo invece di versioni
- **I dati sono il nuovo "Intel Inside"** — il vantaggio competitivo sta nei dati gestiti
- **Esperienze utente ricche** — interfacce reattive grazie ad AJAX

Trade-off: pagine meno leggibili e linkabili da macchina (contenuti generati dinamicamente).

Storia di AJAX

1999 — Microsoft, Internet Explorer 5

- Introdotto come componente ActiveX con il nome *XMLHTTP*: permette al browser di inviare richieste HTTP al server senza ricaricare la pagina
- Pensato originariamente per Outlook Web Access

2002 — Mozilla, standardizzazione

- Mozilla implementa un'interfaccia equivalente con il nome *XMLHttpRequest*, accessibile direttamente da JavaScript senza ActiveX
- Adottata da Safari, Opera e poi dagli altri browser; in seguito standardizzata dal W3C

2005 — Jesse James Garrett conia il termine "Ajax"

- Nell'articolo *Ajax: A New Approach to Web Applications* (febbraio 2005)
- Acronimo di **Asynchronous JavaScript and XML**: dà un nome a una pratica già diffusa, accelerandone l'adozione

Utilizzi del paradigma AJAX

Validazione in tempo reale dei form

- I dati che richiedono un controllo lato server vengono validati mentre l'utente compila, prima dell'invio
- *Es. verifica disponibilità username durante la registrazione*

Auto-completamento

- Una porzione del form viene completata dinamicamente mentre l'utente digita
- *Es. suggerimenti di Google Search, indirizzi su Google Maps*

Operazioni master-detail

- In risposta a un'azione dell'utente, la pagina recupera informazioni più dettagliate senza ricaricarsi
- *Es. tabella che espande i dettagli di una riga al click, anteprima email in Gmail*

Aggiornamento incrementale e live update

- Caricamento di nuovi contenuti o aggiornamenti dal server in background, senza interazione esplicita
- *Es. infinite scroll di feed social, notifiche in tempo reale, dashboard di monitoraggio*

Componenti di AJAX

AJAX combina cinque componenti standard del web:

HTML (o XHTML) e CSS

- Struttura e presentazione delle informazioni

Document Object Model (DOM)

- Rappresentazione ad albero della pagina, navigabile e modificabile da codice per aggiornamenti dinamici

XML e XSLT

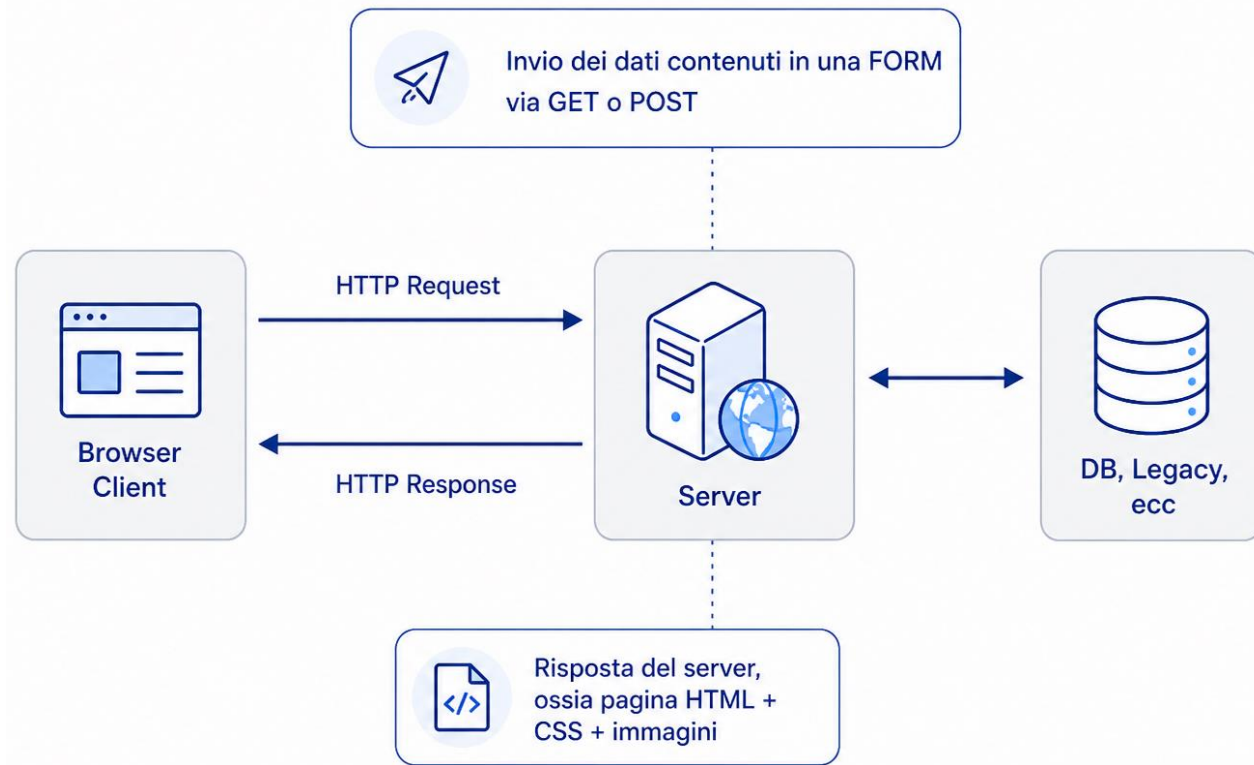
- Formato originario per scambio e trasformazione dei dati tra client e server

L'oggetto XMLHttpRequest

- API del browser per il recupero dei dati dal server in modo **asincrono**

JavaScript

- Il "collante" che orchestra eventi, DOM e richieste XHR



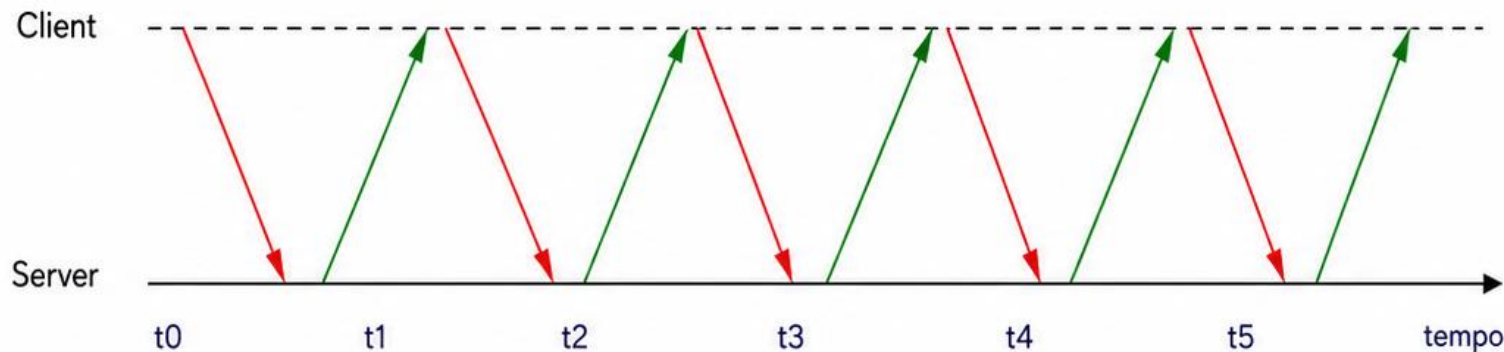
Architettura Web classica

In alto: ciclo richiesta/risposta

Il browser invia una richiesta HTTP (form via GET o POST). Il server interroga il database e restituisce un'intera pagina HTML con CSS e immagini.

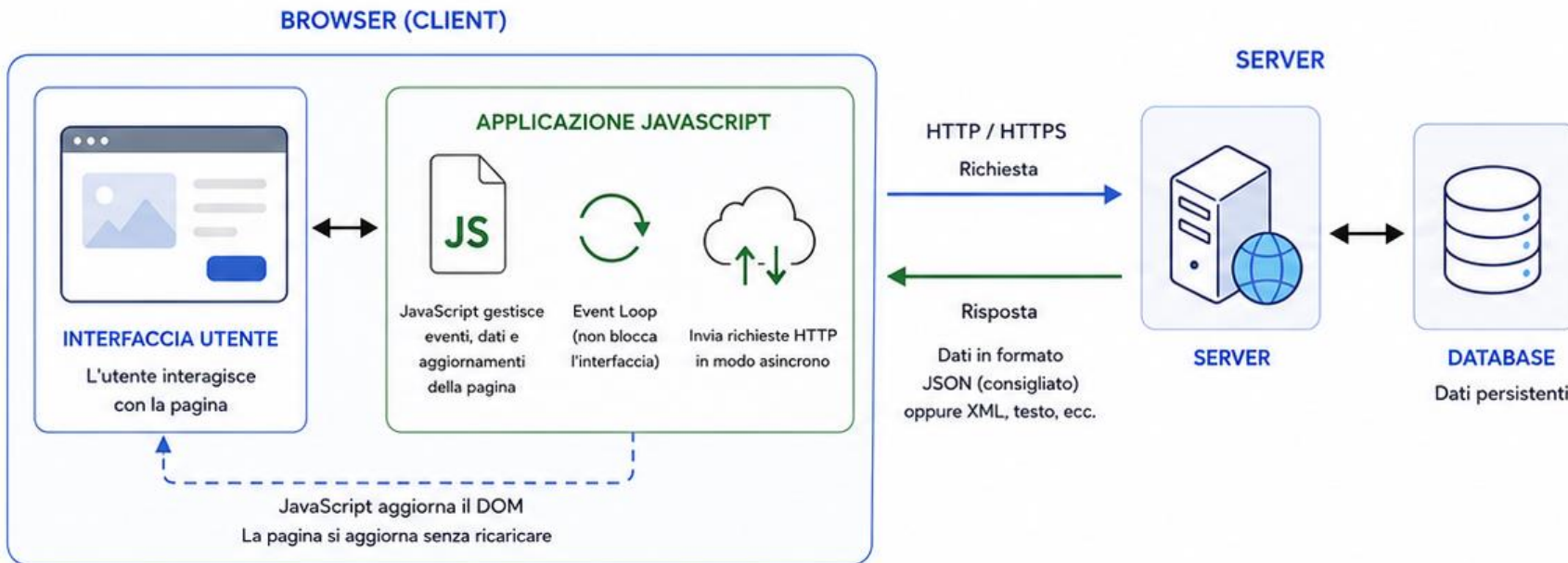
In basso: andamento nel tempo

Ogni interazione produce un ciclo completo Request → Response. Il client resta bloccato in attesa: l'utente non può fare altro finché la pagina non si è ricaricata.



Risultato: esperienza utente discontinua e traffico di rete sproporzionato rispetto all'informazione effettivamente nuova.

Architettura Web con AJAX

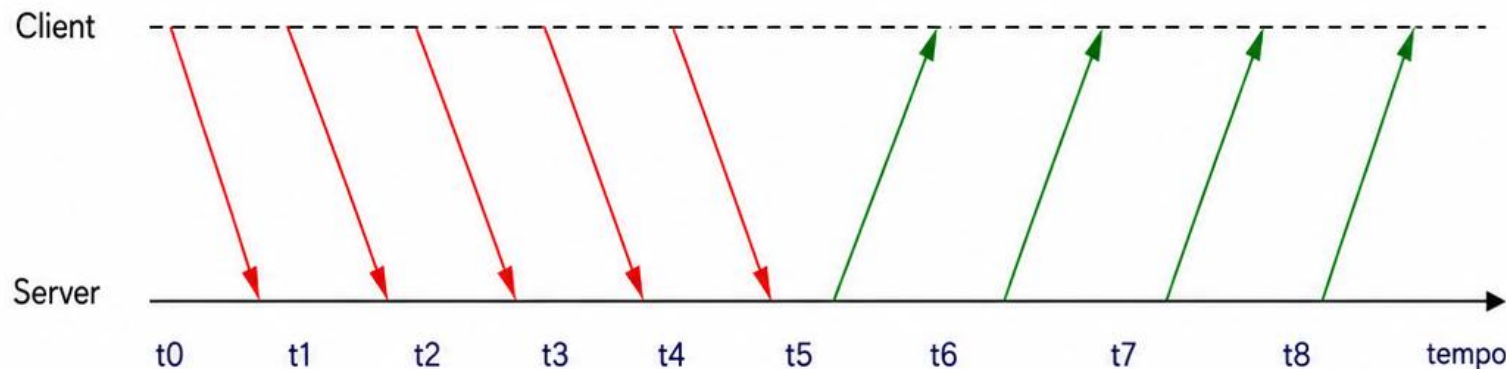


L'AJAX come intermediario

Tra l'interfaccia utente e il server si inserisce un motore JavaScript: l'UI dialoga con l'engine in locale (HTML+CSS), l'engine parla col server in HTTP scambiando solo dati (XML o JSON).

Asincrono

Più richieste possono essere inviate senza attendere le risposte. L'utente può continuare ad interagire mentre le risposte arrivano quando sono pronte e solo i dati necessari vengono aggiornati nella pagina.



Trasmissione dei dati

Lo scambio di dati tra client e server è il punto critico delle applicazioni web interattive: dimensione del payload, parsing e compatibilità influenzano direttamente le prestazioni percepite.

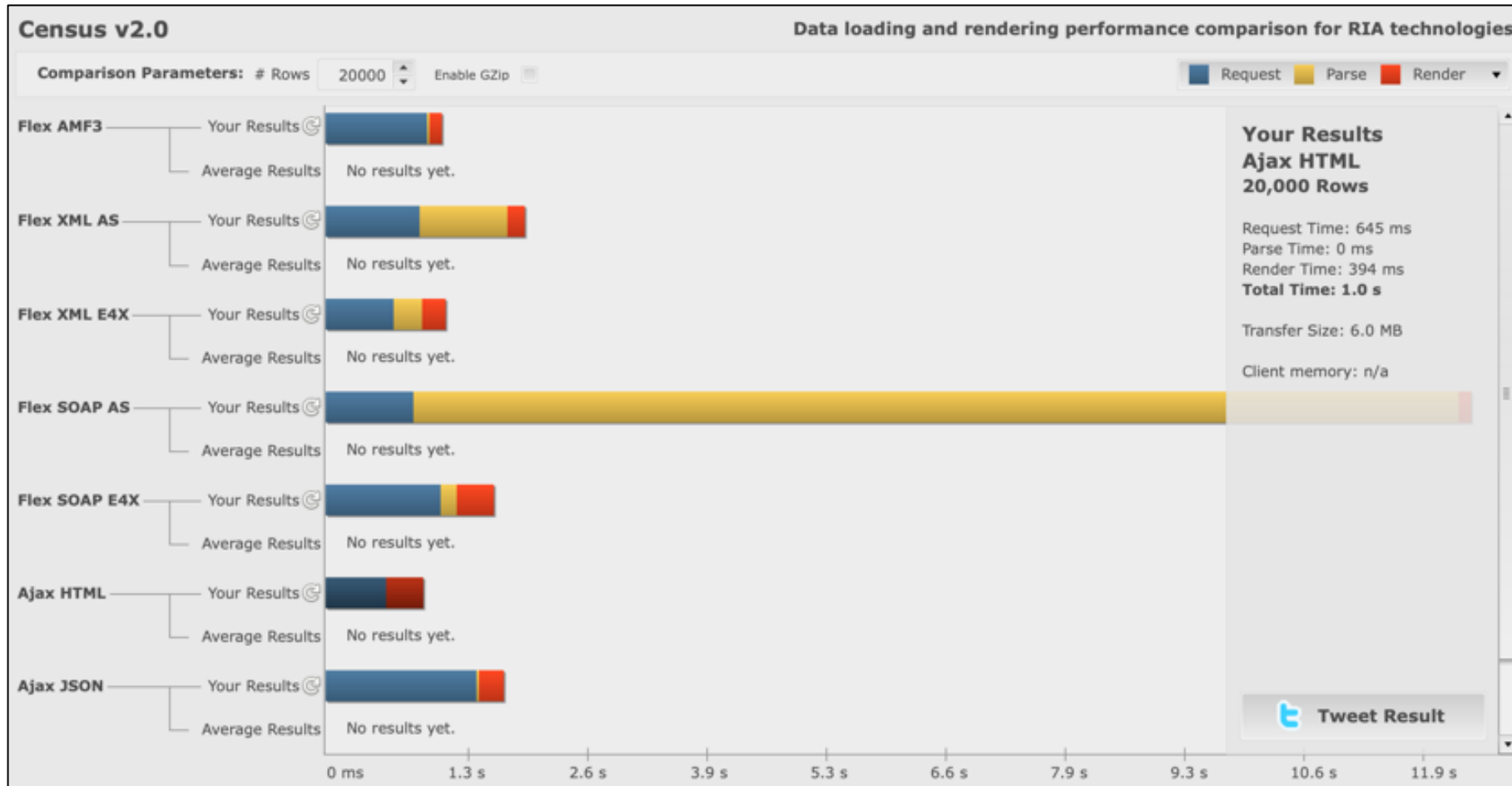
Formati e protocolli

- **JSON** (JavaScript Object Notation): standard de-facto, compatto, parsing nativo nel browser
- **REST** + JSON: pattern dominante per le API web moderne
- **GraphQL**: alternativa che permette al client di richiedere solo i campi che gli servono
- **XML**: formato originario di AJAX, oggi raro nelle web app ma diffuso in contesti enterprise
- **SOAP** e **XML-RPC**: storici, sopravvivono in ambito legacy

Quattro fasi che consumano tempo

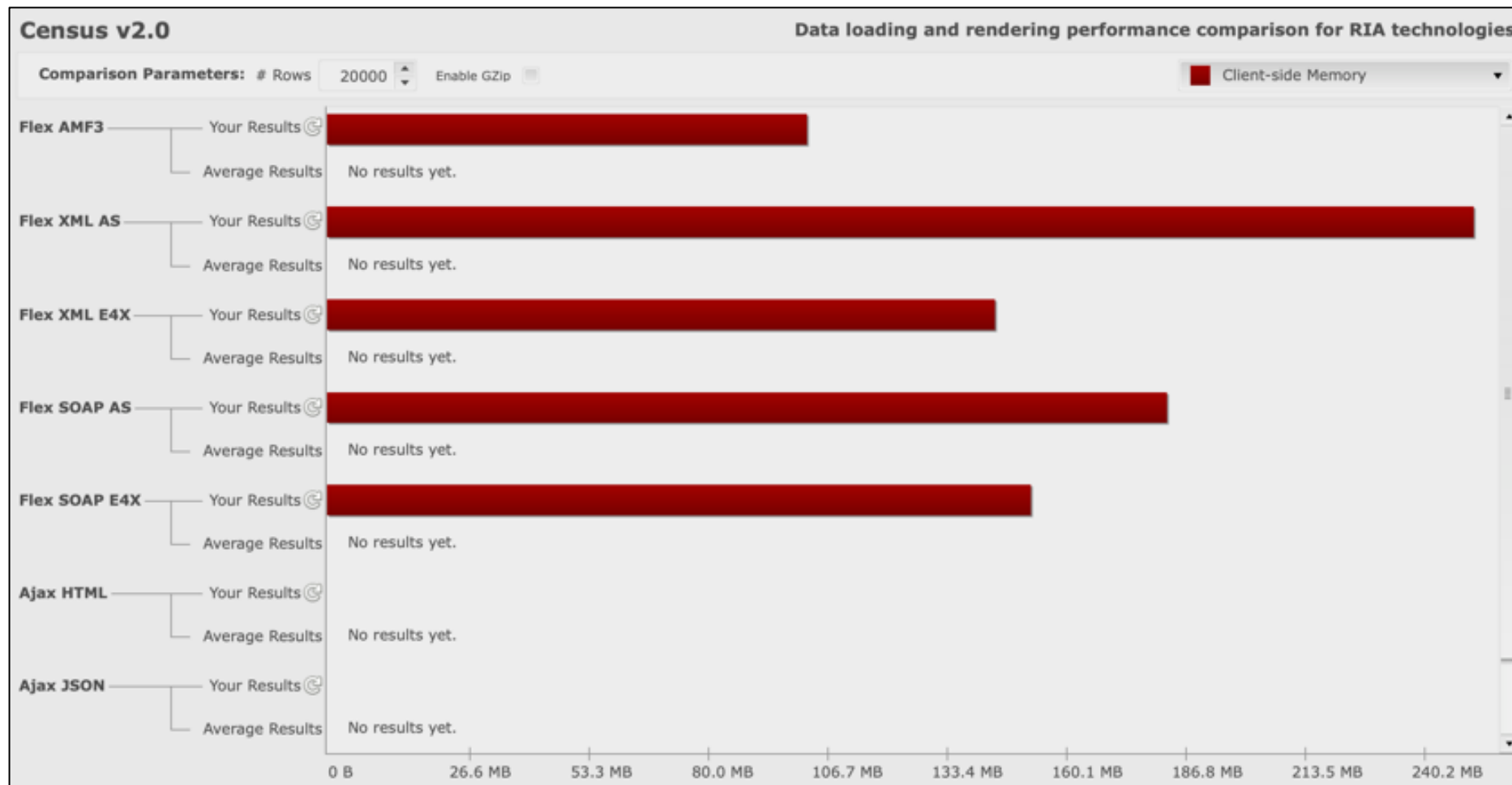
- Serializzazione dei dati lato server
- Trasferimento sulla rete (la dimensione del payload pesa direttamente)
- Parsing lato client
- Aggiornamento del DOM e ridisegno dell'interfaccia

Performance comparison



<http://census2.jamesward.com/>
run: May 7, 2019

Performance comparison



<http://census2.jamesward.com/>

run: May 7, 2019

Modello event-driven del browser

DHTML

Struttura di una pagina HTML5

Una pagina HTML5 ben formata ha una struttura minima fissa. Lo scheletro è composto da quattro elementi annidati:

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <meta charset="UTF-8">
    <title>Pagina di esempio</title>
  </head>
  <body>
    <!-- contenuto, UI, input/output -->
    <script src="app.js" defer></script>
  </body>
</html>
```

Buone pratiche

- Il codice JS in file esterni (separation of concerns), incluso con **defer**: il browser scarica lo script in parallelo all'HTML e lo esegue dopo il parsing
- In alternativa: **type="module"** per usare i moduli ES6 (import/export)

Domande chiave: come vengono visualizzati i risultati? Come comunicano HTML e JavaScript?

JavaScript

Cos'è

- Linguaggio di scripting multi-paradigma (orientato agli oggetti basato su prototipi, funzionale, imperativo) e **dinamicamente tipizzato**: i valori hanno un tipo, le variabili no
- Eseguito da un engine integrato nel browser (V8 in Chrome, SpiderMonkey in Firefox); engine moderni usano compilazione *just-in-time*, non sola interpretazione
- Node.js permette l'esecuzione anche lato server

Ruolo nel pattern AJAX

- Rende dinamiche le pagine HTML modificandone comportamento e visualizzazione tramite il DOM
- Coordina le richieste XMLHttpRequest e gestisce le risposte del server
- Gestisce dati in formato testo, XML e — oggi prevalente — **JSON**

Approfondimenti

- developer.mozilla.org/docs/Learn/JavaScript — tutorial e reference (MDN)
- TypeScript: superset di JavaScript con tipizzazione statica

Visualizzare dati con JavaScript

JavaScript non ha funzioni native di stampa: per "mostrare" qualcosa si scrive nel DOM, in una finestra di dialogo o nella console.

1. Scrivere nel DOM (modo principale)

- **element.textContent** — preferito: inserisce solo testo, sicuro contro XSS
- **element.innerHTML** — interpreta markup HTML; usare solo con contenuto fidato (rischio **XSS** con input utente)

```
document.getElementById("demo").textContent = "Hello world!";
```

2. Console del browser (debug)

- **console.log()** stampa nella DevTools console; varianti utili: **.warn()**, **.error()**, **.table()**

3. Finestre di dialogo (uso limitato)

- **alert()**, **confirm()**, **prompt()** — bloccano l'interfaccia, usare solo per prototipi o casi eccezionali

Antipattern: document.write()

- Se chiamato dopo il caricamento, sostituisce l'intera pagina. Sconsigliato da MDN, da non usare nel codice moderno.

Funzioni ed eventi

Funzioni

- Blocco di codice riutilizzabile, eseguito quando viene **invocato**
- Tre modi di invocazione: chiamata diretta dal codice, in risposta a un **evento**, automaticamente (IIFE — oggi rara grazie ai moduli ES6)

Eventi: cosa accade nella pagina

- Click, input da tastiera, submit di un form, caricamento pagina, scroll, mouseover...
- JavaScript collega una funzione (*handler*) all'evento, e quel codice viene eseguito quando l'evento si verifica

Modo moderno: `addEventListener`

```
const btn = document.getElementById("saluta");  
btn.addEventListener("click", () => alert("Ciao!"));
```

- Più handler sullo stesso evento, codice separato dall'HTML, possibilità di rimuovere il listener

Da evitare: **handler inline** `onclick="..."`

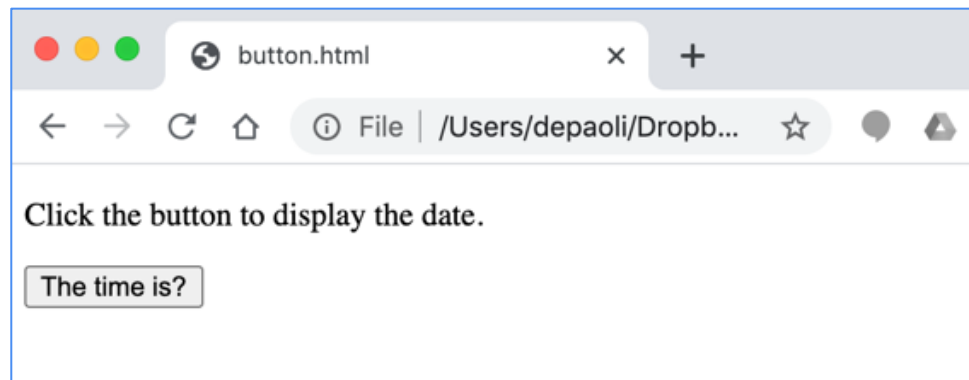
- Mescola logica e markup, un solo handler per evento, difficile da testare e mantenere

Riferimento eventi: developer.mozilla.org/docs/Web/Events

Esempio: dal click al DOM

- Nessun dato in input, solo passaggio di controllo tramite chiamate di funzione

```
1. <!DOCTYPE html>
2. <html>
3. <body>
4.   <p>Fai clic sul pulsante per visualizzare la data.</p>
5.   <input type="button" value="The time is?" onclick="displayDate()">
6.   <p id="demo"></p>
7.   <script>
8.     function displayDate() {
9.       document.getElementById("demo").innerHTML = Date();
10.    }
11.  </script>
12. </body>
13. </html>
```



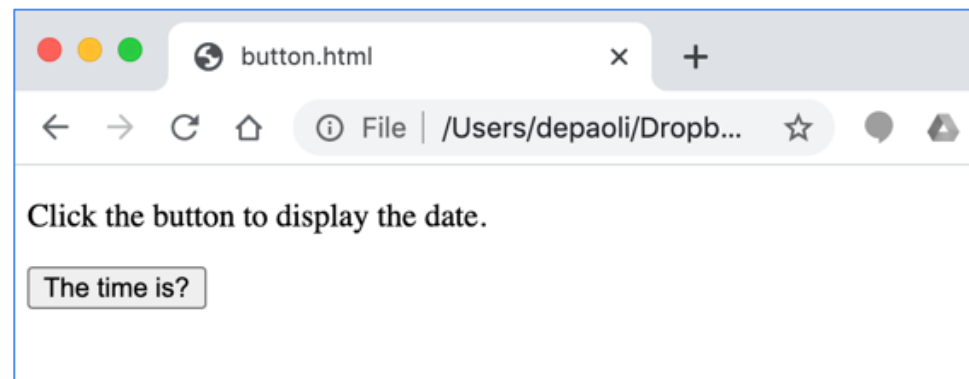
Esempio: dal click al DOM — ① il trigger

- Nessun dato in input, solo passaggio di controllo tramite chiamate di funzione

```
1. <!DOCTYPE html>
2. <html>
3. <body>
4.   <p>Fai clic sul pulsante per visualizzare la data.</p>
5.   <input type="button" value="The time is?" onclick="displayDate()">
6.   <p id="demo"></p>
7.   <script>
8.     function displayDate() {
9.       document.getElementById("demo").innerHTML = Date();
10.    }
11.  </script>
12. </body>
13. </html>
```

① L'attributo onclick

Registra una funzione handler sull'elemento HTML: quando l'utente clicca, il browser esegue automaticamente **displayDate()**.



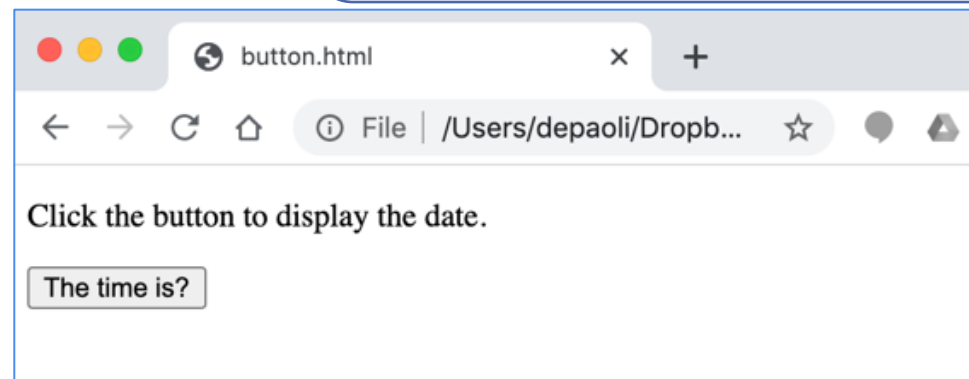
Esempio: dal click al DOM — ② la funzione

- Nessun dato in input, solo passaggio di controllo tramite chiamate di funzione

```
1. <!DOCTYPE html>
2. <html>
3. <body>
4.   <p>Fai clic sul pulsante per visualizzare la data.</p>
5.   <input type="button" value="The time is?" onclick="displayDate()">
6.   <p id="demo"></p>
7.   <script>
8.     function displayDate() {
9.       document.getElementById("demo").innerHTML = Date();
10.    }
11.  </script>
12. </body>
13. </html>
```

② La funzione handler

Definita dentro `<script>`: cerca nel DOM l'elemento con id `"demo"` tramite `document.getElementById`.



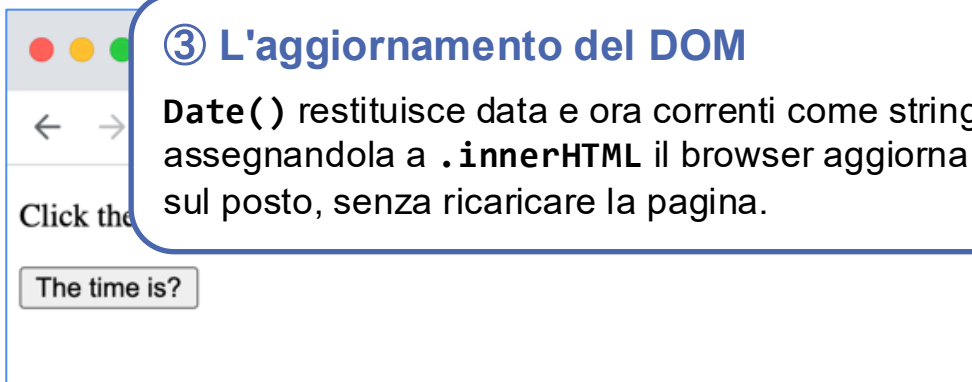
Esempio: dal click al DOM — ③ l'aggiornamento

- Nessun dato in input, solo passaggio di controllo tramite chiamate di funzione

```
1. <!DOCTYPE html>
2. <html>
3. <body>
4.   <p>Fai clic sul pulsante per visualizzare la data.</p>
5.   <input type="button" value="The time is?" onclick="displayDate()">
6.   <p id="demo"></p>
7.   <script>
8.     function displayDate() {
9.       document.getElementById("demo").innerHTML = Date();
10.    }
11.   </script>
12. </body>
13. </html>
```

③ L'aggiornamento del DOM

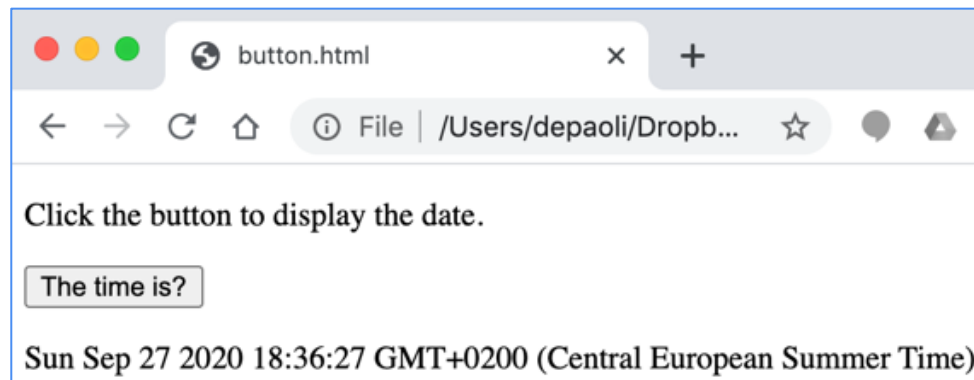
Date() restituisce data e ora correnti come stringa; assegnandola a **.innerHTML** il browser aggiorna l'UI sul posto, senza ricaricare la pagina.



Esempio: dal click al DOM

- Nessun dato in input, solo passaggio di controllo tramite chiamate di funzione

```
1. <!DOCTYPE html>
2. <html>
3. <body>
4.   <p>Fai clic sul pulsante per visualizzare la data.</p>
5.   <input type="button" value="The time is?" onclick="displayDate()">
6.   <p id="demo"></p>
7.   <script>
8.     function displayDate() {
9.       document.getElementById("demo").innerHTML = Date();
10.    }
11.   </script>
12. </body>
13. </html>
```



http://www.w3schools.com/jsref/dom_obj_event.asp

Modelli di esecuzione a confronto

Tre paradigmi differenti rispetto a chi guida il flusso di controllo e quando il programma termina.

Sequenziale / batch

Flusso

- Il programma decide cosa fare e quando
- Esegue input → elabora → output → termina

Caratteristiche

- Terminazione esplicita
- Nessuna interazione durante l'esecuzione

Esempi

- *Compilatore, script di build, job batch*

Interattivo

Flusso

- Il programma chiede input, attende, elabora
- L'utente risponde a richieste predefinite

Caratteristiche

- Controllo guidato dal programma
- Sequenza prevedibile di passi

Esempi

- *CLI a prompt, wizard, REPL*

Reattivo (event-driven)

Flusso

- Il programma reagisce a eventi esterni
- Resta in attesa, non termina

Caratteristiche

- Controllo guidato dall'ambiente
- Asincrono, non deterministico

Esempi

- *GUI, pagine web, server, embedded*

Sistemi reattivi

“... which we call **reactive systems**, are those that are repeatedly prompted by the outside world, and their main role is to **maintain an ongoing interaction with their environment**, rather than to produce some final result on termination.”

— Harel & Pnueli, 1985



Un **sistema reattivo** mantiene un'interazione continua con il suo ambiente, rispondendo a stimoli esterni con tempi e modalità dettati dall'ambiente stesso.

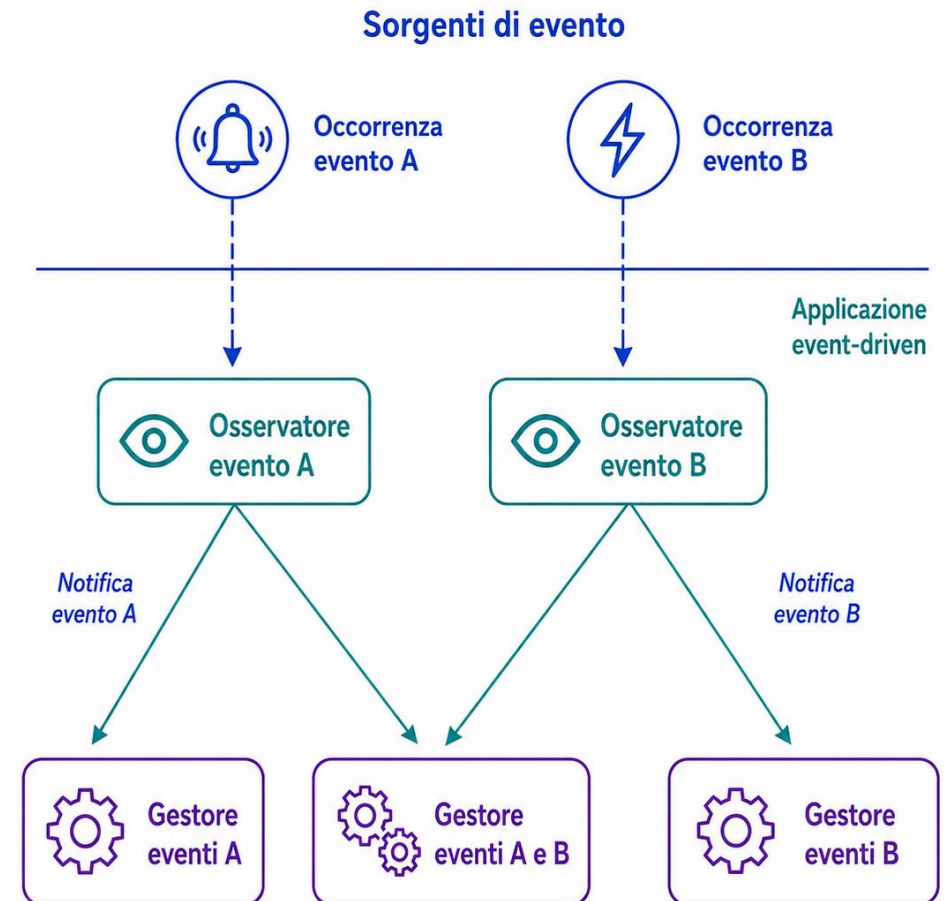
Caratteristiche fondamentali

- Non-terminazione — il sistema resta vivo, in attesa del prossimo evento
- **Asincronia** — gli eventi arrivano in tempi non prevedibili a priori
- **Concorrenza** — più sorgenti di eventi attive simultaneamente

Non è possibile identificare staticamente un flusso di controllo unitario

Il programma principale (main) si limita a *inizializzare* l'applicazione, istanziando gli *osservatori* e associandovi gli opportuni *handler* (*gestori*)

L'associazione *osservatore-gestore* può essere dinamica



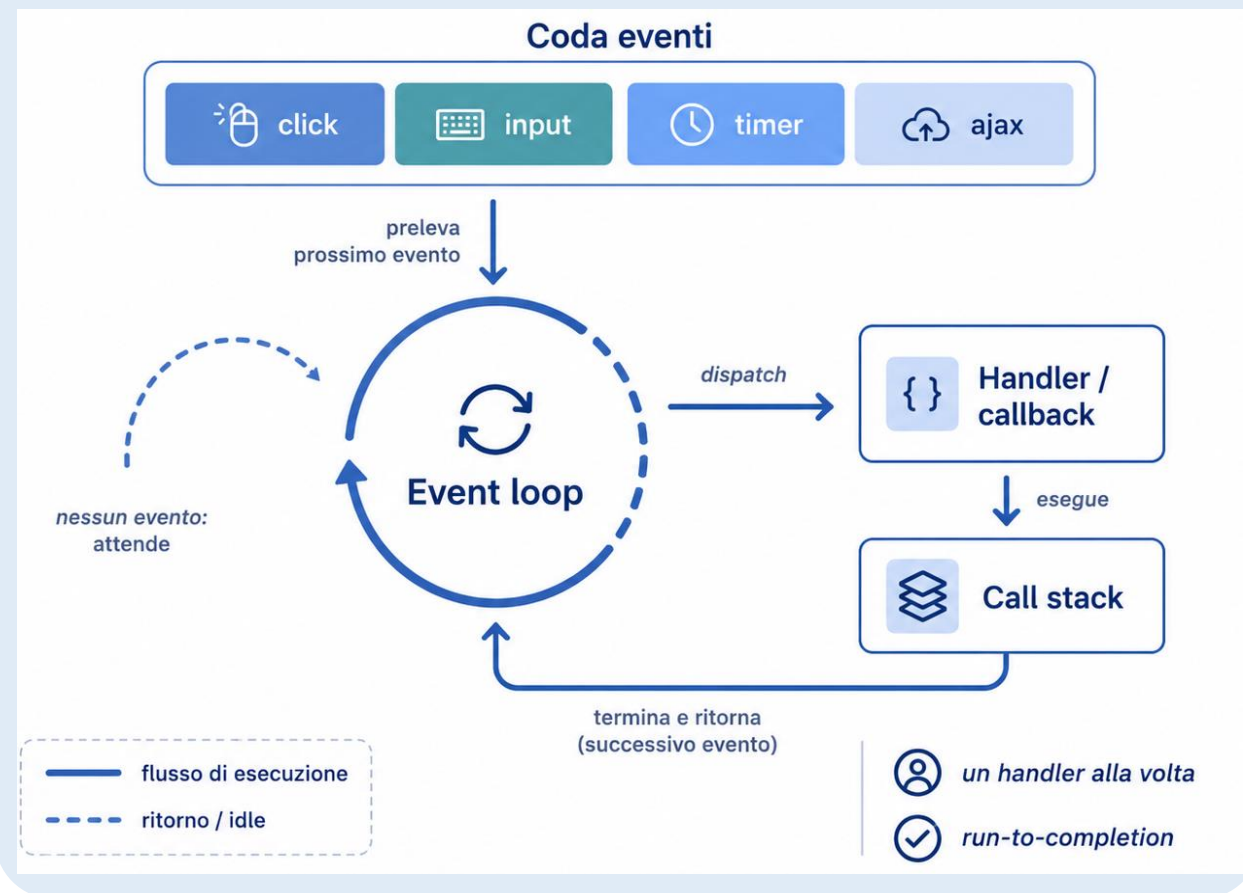
Event Loop

Modello di esecuzione

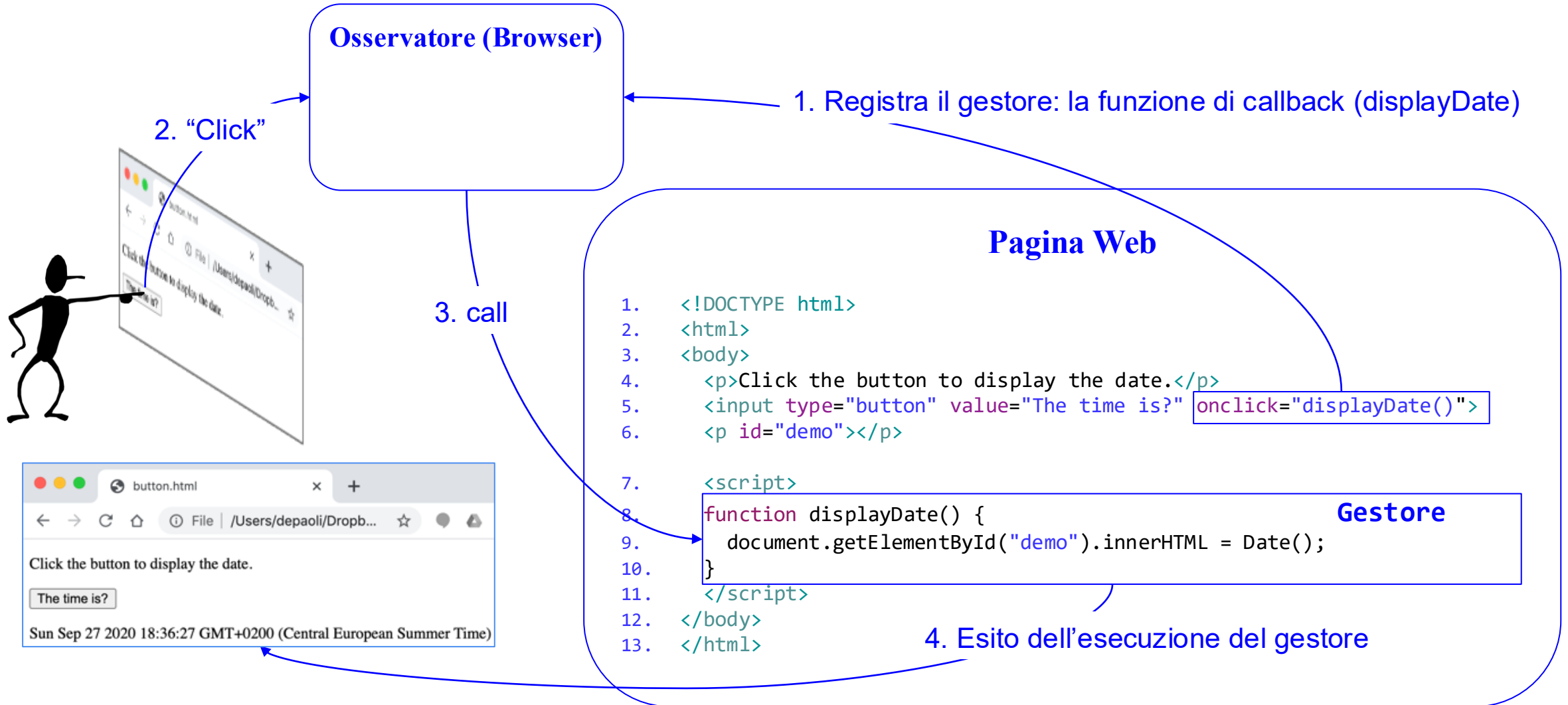
- **Single-threaded** (in JavaScript): un solo handler alla volta
- **Run-to-completion** : ogni handler termina prima del successivo
- Conseguenza: handler lunghi bloccano l'intera UI

Event loop

Ciclo che preleva eventi dalla coda e invoca l'handler associato.



Gestione eventi HTML



addEventListener()

Associa uno o più gestori di eventi (event handlers) a qualsiasi elemento del DOM.

```
element.addEventListener(event, handler, options);
```

1° tipo di evento

- Stringa: "click", "mousedown", "keyup"...
- Senza prefisso "on": "click", non "onclick"

2° handler

- Funzione anonima oppure nome di funzione (senza parentesi)
- Riceve l'oggetto Event come argomento

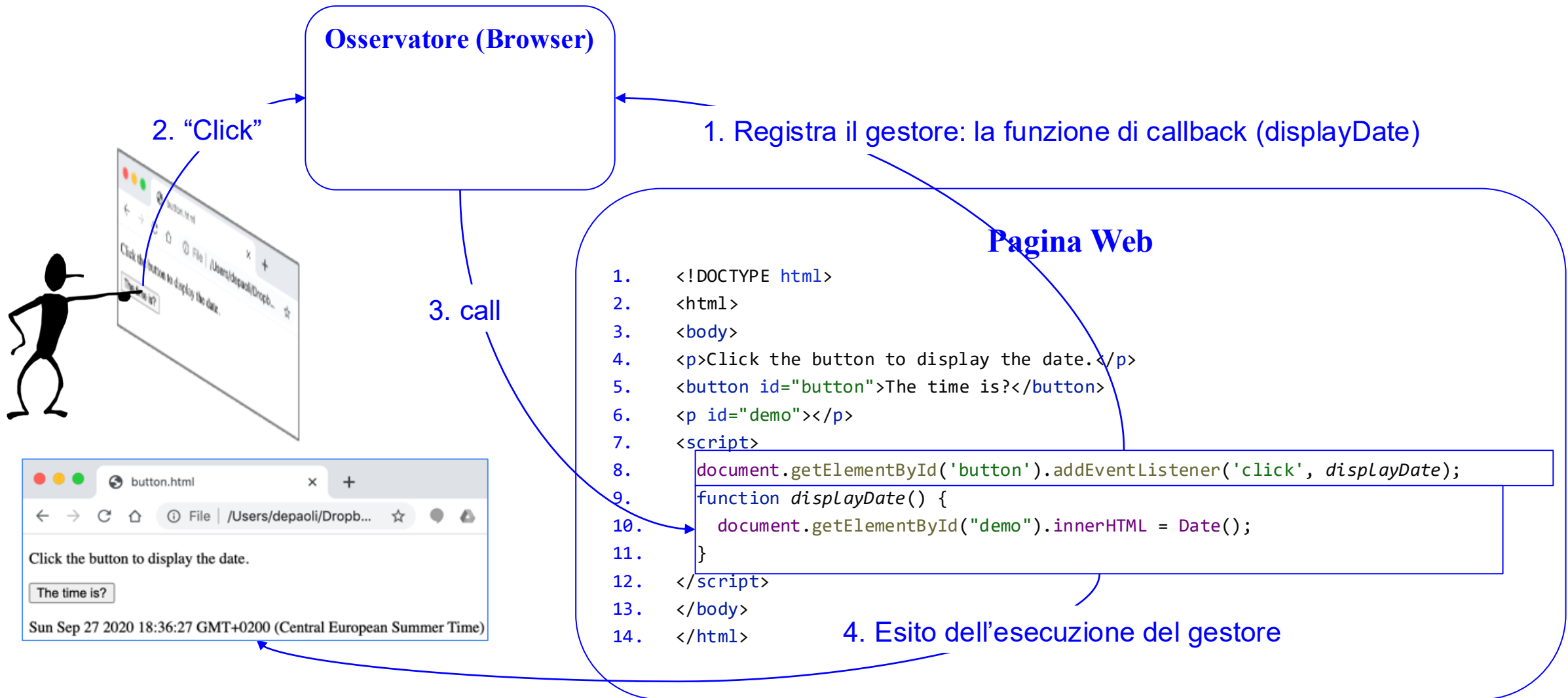
3° (opzionale) — options

- { once: true }, { capture: true }

Esempio

```
1. <!DOCTYPE html>
2. <html>
3. <body>
4.   <button id="saveBtn">Salva</button>
5.   <script>
6.     const btn = document.getElementById("saveBtn");
7.
8.     // Handler con funzione anonima
9.     btn.addEventListener("click", function (e) {
10.       console.log("Cliccato!", e.target);
11.     });
12.
13.     // Handler nominato + options
14.     function onSave(e) { /* ... */ }
15.     btn.addEventListener("click", onSave, { once: true });
16.   </script>
17. </body>
18. </html>
```

Gestione eventi con AddEventListener



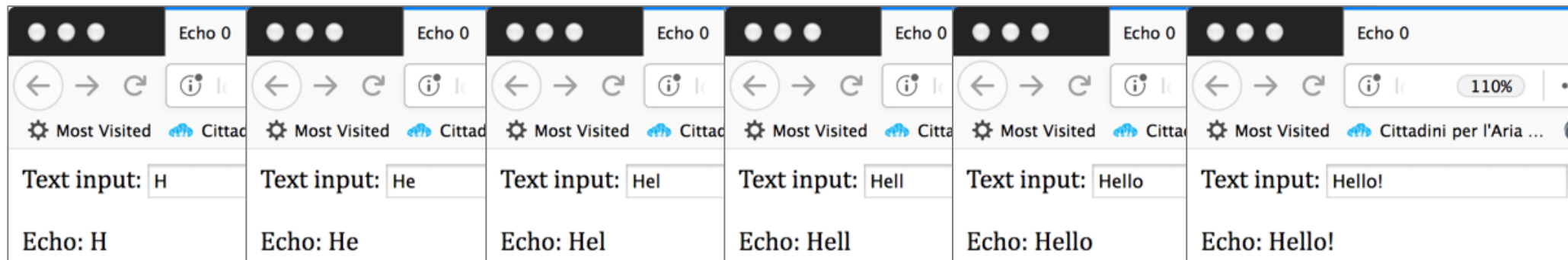
Controllo e input dati

Passaggio di dati tramite chiamate di funzione

```
1. <html>
2. <body>
3. Input di testo: <input type="text" id="txt1" onkeyup="echo1(this.value)">
4. <p>Echo: <span id="demo"></span> </p>
5. <script>
6.   function echo1(str) {
7.     document.getElementById("demo").innerHTML = str;
8.   }
9. </script>
10. </body>
11. </html>
```

On release-button event ...

this = l'elemento **<input>** su cui scatta l'evento;
.value = il testo digitato



Controllo e input dati

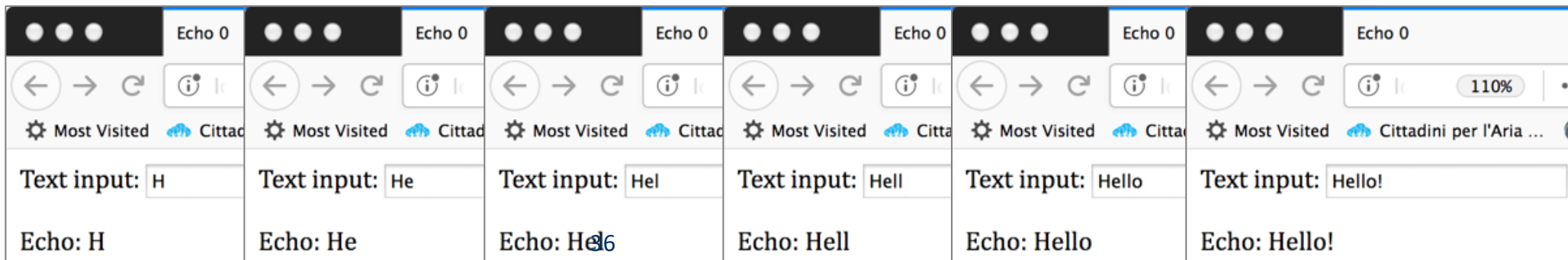
■ Recupero di dati dall'HTML

```
1. <html>
2. <title> Echo 1 </title>
3. <body>
4. <form action="">
5.   Input di testo: <input type="text" id="txt1">
6. </form>
7. <p>Echo: <span id="demo"></span></p>

1. <script>
2.   const input = document.getElementById("txt1");
3.   const output = document.getElementById("demo");
4.   input.addEventListener("input", () => {
5.     output.textContent = input.value;
6.   });
7. </script>
8. </body>
9. </html>
```

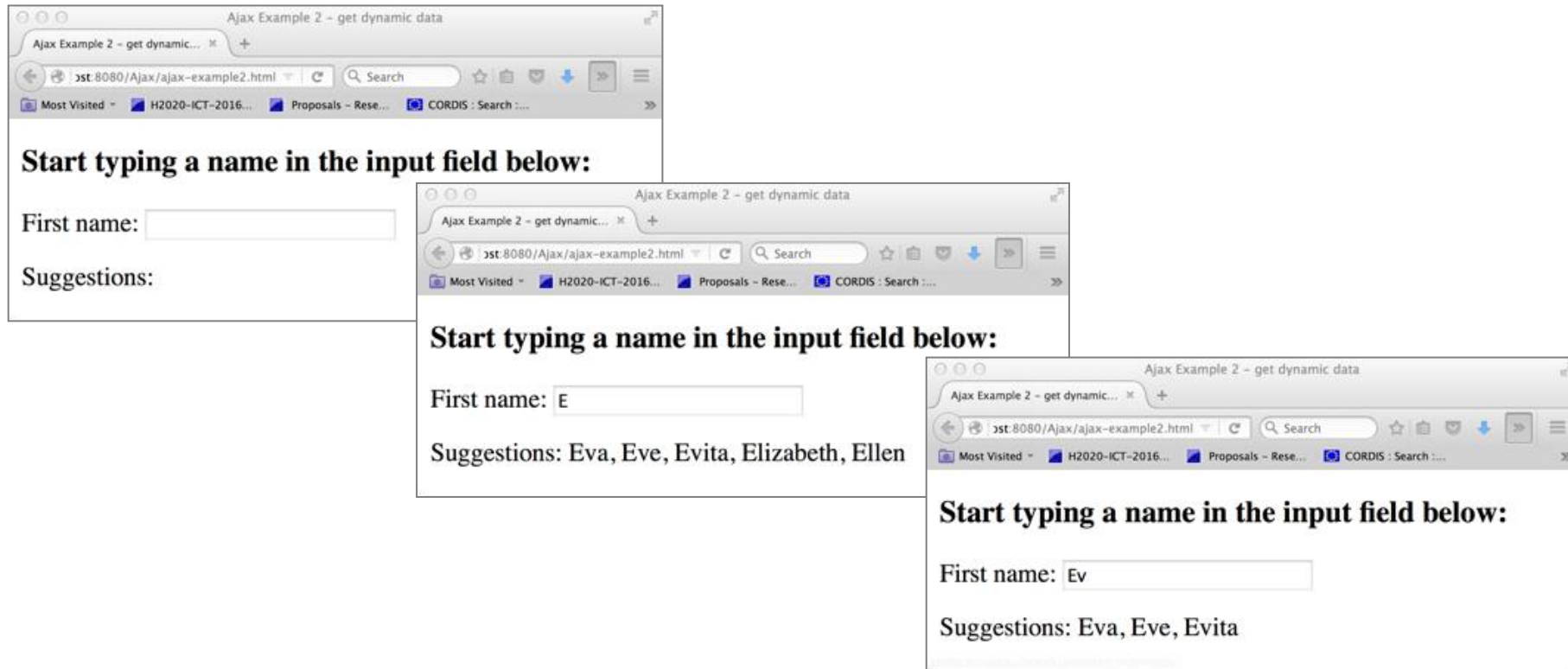
Miglioramenti rispetto alla versione precedente

- *Nessun JavaScript inline*
Il comportamento è separato dall'HTML (onkeyup="...").
- *Evento "input" invece di "keyup"*
Gestisce anche incolla, drag&drop, autocompletamento e input mobile.
- *textContent invece di innerHTML*
Più sicuro ed efficiente: non interpreta HTML.
- *Variabili con const*
Evita riassegnazioni accidentali. Codice più leggibile.
- *addEventListener*
Approccio moderno, flessibile e componibile.
- *Sintassi ES6+*
Uso di arrow function e codice più compatto.



L'esempio con i suggerimenti

L'esempio seguente mostra come una funzione JavaScript può essere inclusa in una pagina web per fornire suggerimenti mentre l'utente digita caratteri in un campo di input



La pagina HTML

- Quando l'utente digita un carattere nel campo di input, viene eseguita una funzione chiamata "showHint()" viene eseguita.
- La funzione è attivata dall'evento `onkeyup` event.
- Il tag `<span>` viene usato per raggruppare elementi inline in un documento e aggiungere un hook a una parte del testo o del documento.

```
1. <html>
2. <head>
3. </head>
4. <body>

5. <h3>Inizia a digitare un nome nel campo di input qui sotto:</h3>
6. <form>
7.   Nome: <input type="text" id="txt1" onkeyup="showHint(this.value)">
8. </form>
9. <p>Suggerimenti: <span id="txtHint"></span></p>

10. <script>
11.   function showHint(str) ...
12. </script>

13. </body>
14. </html>
```

La funzione JavaScript

L'esempio Hint implementato solo lato client

```
1. <script>
2. function showHint(str) {
3.     var names = ["Anna", "Brittany", "Cinderella", "Diana", "Eva", "Fiona", "Gunda", "Hege", "Inga", "Johanna", "Kitty",
4.     "Linda", "Nina", "Ophelia", "Petunia", "Amanda", "Raquel", "Cindy", "Doris", "Eve", "Evita", "Sunniva", "Tove", "Unni",
5.     "Violet", "Liza", "Elizabeth", "Ellen", "Wenche", "Vicky"];
6.     var hint = "";
7.     if (str.length == 0) {
8.         document.getElementById("txtHint").textContent = "";
9.         return;
10.    }
11.    // lookup all hints from array if length of str>0
12.    for (i = 0; i < names.length; i++) {
13.        if (names[i].indexOf(str) == 0) {
14.            if (hint.length == 0)
15.                hint = names[i];
16.            else
17.                hint = hint + ", " + names[i];
18.        }
19.    }
20.    document.getElementById("txtHint").textContent = hint;
21. }
22. </script>
```

Dichiaro un elenco vuoto, che conterrà i nomi con le iniziali cercate

Se la stringa di caratteri ricevuta è vuota non faccio nulla

Scelgo il nome che inizia con la stringa di caratteri ricevuta

La prima volta inserisco il nome trovato come primo elemento dell'elenco

La seconda volta aggiungo il nome trovato all'elenco esistente preceduto da una virgola

The `indexOf()` method returns the position of the first occurrence of a specified value in a string.
This method returns -1 if the value to search for never occurs.

Note: The `indexOf()` method is case sensitive.

https://www.w3schools.com/jsref/jsref_indexof.asp

Riferimenti

- Alcuni esempi sono tratti da:
<http://www.w3schools.com/Ajax/default.asp>
- Ulteriori informazioni su JavaScript:
<http://www.w3schools.com/js/default.asp>
- Dettagli sugli eventi:
http://www.w3schools.com/js/js_events_examples.asp
- Risorse Mozilla
<https://developer.mozilla.org/en-US/docs/Web>
- Fonti in italiano
<http://www.html.it/guide/guida-ajax/>
<http://www.html.it/guide/guida-javascript-di-base/>
- CSS
https://developer.mozilla.org/en-US/docs/Web/Guide/CSS/Getting_started/JavaScript
https://developer.mozilla.org/it/docs/Conoscere_i_CSS
<http://www.html.it/pag/32401/html-css-e-javascript/>